

A. Architecture

S-O-L-I-D Prensipleri

Neden?	SOLID, kodunu küçük, tek sorumlu, birbirinden bağımsız parçalara ayırır. Yeni özellik eklerken mevcut kodu bozmadan genişletebilirsin. Takım çalışmasında merge conflict'leri azalır, test yazmak kolaylaşır.
Kullanmasam?	Sınıflar şişer, bir yerde yapılan değişiklik başka yerleri kırar. Zamanla 'spaghetti code' oluşur; bir silah değiştirmek için 5 farklı sınıfa dokunmak zorunda kalırsın.
Nerede?	Enemy, Player, Weapon, Ability System, UIManager sınıflarının tasarımında. Her yeni sistem yazmadan önce 'Bu sınıfın tek sorumluluğu ne?' sorusunu sor.

Oyun İçi Kullanım Örneği

```
// S - Single Responsibility: Sadece hasar hesaplar
public class DamageCalculator {
    public float Calculate(float baseDmg, float multiplier) => baseDmg * multiplier;
}

// O - Open/Closed: Silah tipi eklemek için mevcut kodu değiştirme
public abstract class WeaponBase {
    public abstract void Fire();
}
public class Bow : WeaponBase { public override void Fire() { /* ok atar */ } }
public class Staff : WeaponBase { public override void Fire() { /* büyü yapar */ } }

// L - Liskov: Alt sınıflar üst sınıfın yerine geçebilmeli
// I - Interface Segregation: IMovable, IDamageable ayrı ayrı
// D - Dependency Inversion: Soyutlamaya bağlan, somut sınıfa değil
```

A. Architecture

Abstraction (Soyutlama)

Neden?	Kullanıcı kodunun implementasyon detaylarından haberdar olmamasını sağlar. Servis değiştiğinde (örn. farklı ses motoru), interface'i kullanan kod değişmez. Mocklamayı kolaylaştırır.
Kullanmasam?	Her sistem birbirine sıkıca bağlıdır (tightly coupled). Ses sistemini değiştirmek istersen, onu kullanan 50 yeri güncellemeniz gerekir.
Nerede?	IAudioService, ISaveService, IInputProvider, IEnemyAI gibi interface tanımlamalarında kullan.

Oyun İçi Kullanım Örneği

```
// Interface (abstraction layer)
public interface IAudioService {
    void Play(string clipName);
    void Stop(string clipName);
}

// Concrete implementation
public class UnityAudioService : IAudioService {
    public void Play(string clipName) { /* AudioSource.Play... */ }
    public void Stop(string clipName) { /* ... */ }
}

// Tüketen sınıf sadece interface'i bilir
public class Player {
    private IAudioService audio;
    public Player(IAudioService audio) => _audio = audio;
    void OnJump() => audio.Play("jump sfx");
}
```

A. Architecture

Singleton Pattern

Neden?	Oyun genelinde tek örneği olması gereken sistemler (GameManager, AudioManager, EventBus) için global erişim noktası sağlar. Doğru kullanıldığında scene geçişlerinde varlığını korur.
Kullanmasam?	Aynı servise erişmek için her yerde Find() veya GetComponent() çağırırsın, performans düşer ve null reference hataları artar.
Nerede?	GameManager, AudioManager, SceneLoader, EventBus gibi tek instance gerektiren sistemlerde. Her şeyi singleton yapma, sadece gerçekten global olanlar.

Oyun İçi Kullanım Örneği

```
public class GameManager : MonoBehaviour {
    public static GameManager Instance { get; private set; }

    void Awake() {
        if (Instance != null) { Destroy(gameObject); return; }
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    public void PauseGame() { Time.timeScale = 0f; }
    public void ResumeGame() { Time.timeScale = 1f; }
}

// Kullanım:
// GameManager.Instance.PauseGame();
```

A. Architecture

ScriptableObject Architecture

Neden?	Oyun verilerini (enemy stats, item configs, level data) kod dışında, Inspector'dan düzenlenebilir asset olarak tutar. Designer/balancer koddan bağımsız çalışabilir. Runtime'da clone edilmeden orijinal veriyi korur.
Kullanmasam?	Prefab'lara gömülü değerler karmaşılaşır, değer değiştirmek için prefab açmak gerekir. Aynı veriyi birden fazla yerde tutarsın.
Nerede?	EnemyData, WeaponConfig, LevelConfig, AbilityData, ItemDatabase gibi tüm oyun verisi asset'lerinde.

Oyun İçi Kullanım Örneği

```
[CreateAssetMenu(menuName = "Game/EnemyData")]
public class EnemyData : ScriptableObject {
    public string enemyName;
    public float maxHealth;
    public float moveSpeed;
    public float attackDamage;
    public float spawnWeight;
}

// Enemy verisini SO'dan okur
public class Enemy : MonoBehaviour {
    [SerializeField] private EnemyData _data;
    private float _currentHealth;

    void Start() => currentHealth = _data.maxHealth;
    public void TakeDamage(float dmg) => _currentHealth -= dmg;
}
```

A. Architecture

Factory Pattern

Neden? Nesne oluşturma mantığını merkezi bir yere toplar. Hangi tipin oluşturulacağını runtime'da belirleyebilirsiniz. Object pooling ile kolayca entegre olur.

Kullanmasam? Instantiate çağrılarını her yere dağılır, aynı spawn mantığını farklı yerlerde tekrarlırsın (DRY ihlali) ve değiştirmek zorlaşır.

Nerede? EnemyFactory, ProjectileFactory, VFXFactory, UIElementFactory olarak tüm Instantiate mantığını buraya toplar.

Oyun İçi Kullanım Örneği

```
public class EnemyFactory : MonoBehaviour {
    [SerializeField] private EnemyData[] _enemyTypes;
    [SerializeField] private Transform _spawnParent;

    public Enemy Create(string id, Vector3 pos) {
        var data = Array.Find(_enemyTypes, e => e.name == id);
        var go = Instantiate(data.prefab, pos, Quaternion.identity, _spawnParent);
        var enemy = go.GetComponent<Enemy>();
        enemy.Initialize(data);
        return enemy;
    }
}
```

A. Architecture

Polymorphism (Çok Biçimlilik)

Neden? Farklı düşman tipleri, silahlar veya ability'ler aynı arayüzü uygulayabilir. Yeni tip eklerken mevcut kodu değiştirmezsin; sadece yeni sınıf yazarsın.

Kullanmasam? Switch/if-else zincirleri oluşur. Yeni bir düşman tipi için 10 farklı yeri güncellemeniz gerekir.

Nerede? Enemy AI tipleri, Ability sistemi, Weapon tipleri, UI elementleri, efekt sistemi.

Oyun İçi Kullanım Örneği

```
public abstract class AbilityBase : ScriptableObject {
    public abstract void Activate(PlayerController player);
    public abstract void Deactivate(PlayerController player);
}

public class DashAbility : AbilityBase {
    public float dashForce = 20f;
    public override void Activate(PlayerController p) { /* dash uygula */ }
    public override void Deactivate(PlayerController p) { }
}

public class ShieldAbility : AbilityBase {
    public override void Activate(PlayerController p) { /* kalkan aç */ }
    public override void Deactivate(PlayerController p) { /* kalkan kapat */ }
}

// Oyuncu tüm ability'leri aynı şekilde kullanır:
// foreach (var ability in equippedAbilities) ability.Activate(player);
```

A. Architecture

Base Entity Definitions

Neden? Ortak davranışları (health, damage, death) tek yerde tanımlar, tüm entityler buradan miras alır. Kod tekrarı olmaz, bir değişiklik tüm entityleri etkiler.

Kullanmasam? Her sınıfa ayrı ayrı health sistemi yazarsın. Bir bug düzeltmek için 10 farklı sınıfı güncellemeniz gerekir.

Nerede? Entity (tüm oyun nesnelerinin atası), DamageableEntity, MovableEntity, InteractableEntity.

Oyun İçi Kullanım Örneği

```
public abstract class Entity : MonoBehaviour {
    protected EntityData data;
    protected float currentHealth;
    public bool IsAlive => currentHealth > 0f;

    public virtual void Initialize(EntityData d) {
        data = d; currentHealth = d.maxHealth;
    }
    public virtual void TakeDamage(float amount) {
        currentHealth = Mathf.Max(0, currentHealth - amount);
        if (!IsAlive) OnDeath();
    }
    protected abstract void OnDeath();
}

public class Goblin : Entity {
    protected override void OnDeath() {
        DropLoot();
        VFXFactory.Spawn("death vfx", transform.position);
        gameObject.SetActive(false);
    }
}
```

A. Architecture

Decoupling (Bağımlılığı Azaltma)

Neden?	Sistemler arası doğrudan bağımlılıkları kaldırır. EventBus veya messaging pattern ile komponentler birbirini bilmeden haberleşir. Test, değiştirme ve genişletme kolaylaşır.
Kullanmasam?	Player, Enemy'ye direkt referans tutar. Enemy silindiğinde NullReferenceException alırsın. Modülleri bağımsız test edemezsin.
Nerede?	Event sistemleri (UnityEvents, C# events, ScriptableObject events), Dependency Injection, Service Locator pattern.

Oyun İçi Kullanım Örneği

```
[CreateAssetMenu(menuName="Events/GameEvent")]
public class GameEvent : ScriptableObject {
    private List<GameEventListener> listeners = new();

    public void Raise() {
        for (int i = _listeners.Count - 1; i >= 0; i--)
            _listeners[i].OnEventRaised();
    }
    public void Register(GameEventListener l) => _listeners.Add(l);
    public void Unregister(GameEventListener l) => listeners.Remove(l);
}

// Düşman öldüğünde event fırlatır, diğer sistemleri bilmez
public class Enemy : Entity {
    [SerializeField] private GameEvent _onEnemyDied;
    protected override void OnDeath() => onEnemyDied.Raise();
}
```

B. Object Pooling

Projectile Pooling (Mermi Havuzu)

Neden?	Her ateşlemede Instantiate/Destroy yapmak GC spike'larına ve frame drop'a yol açar. Pool ile mermiler hazır tutulur, sadece aktif/pasif yapılır. 60 fps'de yüzlerce mermi bile sorunsuz çalışır.
Kullanmasam?	Her saniye onlarca Instantiate/Destroy: bellek parçalanması, GC pause'ları ve görünür kekeleme. Mobil platformlarda kritik performans sorunu.
Nerede?	Ok, mermi, büyü topu, lazer segmenti gibi sık Instantiate/Destroy yapılan tüm nesneler.

Oyun İçi Kullanım Örneği

```
public class ProjectilePool : MonoBehaviour {
    [SerializeField] private Projectile _prefab;
    [SerializeField] private int _initialSize = 30;
    private Queue<Projectile> _pool = new();

    void Start() {
        for (int i = 0; i < _initialSize; i++) {
            var p = Instantiate(_prefab);
            p.gameObject.SetActive(false);
            p.Pool = this;
            _pool.Enqueue(p);
        }
    }

    public Projectile Get(Vector3 pos, Quaternion rot) {
        var p = _pool.Count > 0 ? _pool.Dequeue() : Instantiate(_prefab);
        p.transform.SetPositionAndRotation(pos, rot);
        p.gameObject.SetActive(true);
        return p;
    }

    public void Return(Projectile p) {
        p.gameObject.SetActive(false);
        _pool.Enqueue(p);
    }
}
```

B. Object Pooling

Floating Texts (Hasar / Puan Yazısı)

Neden?	+150 hasar, +50 altın gibi floating text'ler sık spawn olur. Pool ile her text nesnesini yeniden kullanırsın. Canvas elementleri için Instantiate maliyeti özellikle yüksektir.
Kullanmasam?	Her hasar darbesinde UI Instantiate edilir, Canvas rebuild tetiklenir. Yoğun savaşlarda ciddi frame düşüşü yaşanır.
Nerede?	Hasar sayıları, kritik vurgu, para/exp kazanma, debuff göstergeleri.

Oyun İçi Kullanım Örneği

```
public class FloatingTextPool : MonoBehaviour {
    [SerializeField] private FloatingText _prefab;
    [SerializeField] private Canvas worldCanvas;
    private Stack<FloatingText> _pool = new();

    public void Show(string text, Vector3 worldPos, Color col) {
        var ft = _pool.Count > 0
            ? _pool.Pop()
            : Instantiate(_prefab, worldCanvas.transform);
        ft.gameObject.SetActive(true);
        ft.Play(text, worldPos, col, () => Return(ft));
    }

    void Return(FloatingText ft) {
        ft.gameObject.SetActive(false);
        _pool.Push(ft);
    }
}

// FloatingTextPool.Instance.Show("-150", enemy.transform.position, Color.red);
```

B. Object Pooling

VFX & Enemy Pooling

Neden? Particle effect ve enemy prefab'ları ağır nesnelerdir. Pooling ile hem CPU hem bellek kullanımı drastik düşer. Unity'nin built-in Pool sistemi (UnityEngine.Pool) production-ready'dir.

Kullanmasam? Her düşman ölümünde Destroy, her VFX'te Instantiate. 50+ düşmanın olduğu sahnede performans çöker.

Nerede? ParticleSystem efektleri (hit, death, explosion), tüm düşman tipleri, çevre objeleri (mermi kovanları vb.).

Oyun İçi Kullanım Örneği

```
using UnityEngine.Pool;

public class VFXPool : MonoBehaviour {
    [SerializeField] private ParticleSystem _hitVFX;
    private ObjectPool<ParticleSystem> pool;

    void Awake() {
        _pool = new ObjectPool<ParticleSystem>(
            createFunc:      () => Instantiate( hitVFX),
            actionOnGet:     ps => ps.gameObject.SetActive(true),
            actionOnRelease: ps => ps.gameObject.SetActive(false),
            actionOnDestroy: ps => Destroy(ps.gameObject),
            defaultCapacity: 20
        );
    }

    public void PlayAt(Vector3 pos) {
        var ps = _pool.Get();
        ps.transform.position = pos;
        ps.Play();
        StartCoroutine(ReturnAfter(ps, ps.main.duration));
    }

    IEnumerator ReturnAfter(ParticleSystem ps, float delay) {
        yield return new WaitForSeconds(delay);
        _pool.Release(ps);
    }
}
```

B. Object Pooling

Sound Pooling

Neden?	Oyun seslerini her seferinde yeni AudioSource ile çalmak maliyet yaratır. Birden fazla aynı ses çalarken channel yönetimi zorunlu hale gelir.
Kullanmasam?	Tek AudioSource ile yönetirsen yeni ses başlayınca önceki kesilir. Çok sayıda AudioSource yaratmak bellek sorunlarına yol açar.
Nerede?	SFX sistemi (hit, shoot, pickup, footstep, UI click), ambient seslerin yönetimi.

Oyun İçi Kullanım Örneği

```
public class AudioPool : MonoBehaviour {
    [SerializeField] private int _poolSize = 16;
    private Queue _sources = new();

    void Awake() {
        for (int i = 0; i < _poolSize; i++) {
            var src = gameObject.AddComponent();
            src.playOnAwake = false;
            _sources.Enqueue(src);
        }
    }

    public void Play(AudioClip clip, float volume = 1f) {
        var src = _sources.Dequeue();
        src.clip = clip;
        src.volume = volume;
        src.Play();
        _sources.Enqueue(src); // Circular queue
    }
}
```

B. Object Pooling

Spawn Rate & Data-Driven Düşman Kontrolü

Neden? Düşman spawn zamanlamasını ve yoğunluğunu ScriptableObject'te tutmak, tasarımcıların kod yazmadan oyun dengesini ayarlamasını sağlar. Wave sistemi curve-driven olabilir.

Kullanmasam? Spawn değerleri koda gömülür. Denge değişikliği için programcı gerekir, her değişiklik yeniden derleme ister.

Nerede? WaveManager, EnemySpawner, DifficultyScaler; SpawnData asset'i ile kontrol.

Oyun İçi Kullanım Örneği

```
[CreateAssetMenu(menuName="Game/WaveData")]
public class WaveData : ScriptableObject {
    public EnemySpawnEntry[] entries;
    public float waveDuration;
    public AnimationCurve spawnRateCurve; // Zamana göre spawn hızı
}

[System.Serializable]
public class EnemySpawnEntry {
    public EnemyData enemyData;
    public float spawnWeight;
    public float startAppearTime; // Dalganın kaçınca saniyesinde çıkar
}

// Spawner interval'i curve'den hesaplar:
// float interval = 1f / _wave.spawnRateCurve.Evaluate(elapsed / _wave.waveDuration);
```

B. Object Pooling → MVC/MVP

MVC/MVP Mimarisi (UI & Logic Ayrımı)

Neden? Model (veri), View (görsel) ve Presenter/Controller (mantık) ayrılınca UI değiştiğinde oyun mantığına, mantık değiştiğinde UI'ya dokunmak gerekmez. Birim test yazmak kolaylaşır.

Kullanmasam? UI sınıfları direkt oyun verisi yönetmeye başlar. Health bar içinde hasar hesabı yapılır. Spaghetti UI kodu test edilemez ve bakımı imkansızdır.

Nerede? HealthBar Presenter, Inventory Presenter, ShopPresenter, HUDPresenter; tüm oyuncu/düşman arayüzleri.

Oyun İçi Kullanım Örneği

```
// Model: Saf veri
public class PlayerModel {
    public float CurrentHealth { get; private set; }
    public float MaxHealth { get; private set; }
    public event Action<float, float> OnHealthChanged;

    public void TakeDamage(float dmg) {
        CurrentHealth = Mathf.Max(0, CurrentHealth - dmg);
        OnHealthChanged?.Invoke(CurrentHealth, MaxHealth);
    }
}

// Presenter: Model'i dinler, View'i günceller
public class HealthBarPresenter : MonoBehaviour {
    [SerializeField] private Slider slider;
    private PlayerModel _model;

    public void Bind(PlayerModel model) {
        model = model;
        _model.OnHealthChanged += UpdateView;
    }

    void UpdateView(float current, float max) => _slider.value = current / max;
}
```

C. Addressables

Memory Unload (Bellek Temizleme)

Neden? Kullanılmayan asset'leri bellekten atmak, özellikle mobilde zorunludur. Addressables Release ile yüklenen her asset'i referans sayısı sıfırlandığında otomatik boşaltabilirsiniz.

Kullanmasam? Boss bölüm texture'ları, bölüm bittikten sonra hâlâ bellekte kalır. Uzun oyun sessionlarında RAM dolar, iOS/Android oyunu kill eder.

Nerede? Level geçişlerinde, boss savaşı sonrası, shop/menu kapanışlarında asset release çalışır.

Oyun İçi Kullanım Örneği

```
private AsyncOperationHandle<Sprite> _handle;

async void LoadBossTexture() {
    handle = Addressables.LoadAssetAsync<Sprite>("boss_dragon");
    await _handle.Task;
    bossImage.sprite = _handle.Result;
}

// Kullanım bitince belleği serbest bırak
void UnloadBossTexture() {
    if (_handle.IsValid())
        Addressables.Release(_handle);
}

// Sahne unload sırasında tetiklenir
void OnDestroy() => UnloadBossTexture();
```

C. Addressables

Lazy Loading

Neden?	Oyun başlangıçta her şeyi yüklemek zorunda değil. İhtiyaç duyulduğunda (lazily) yüklemek, hem ilk açılış süresini hem de anlık bellek kullanımını düşürür.
Kullanmasam?	Tüm asset'leri baştan yüklemek = uzun loading ekranları, yüksek RAM kullanımı ve düşük-end cihazlarda crash.
Nerede?	Shop item görselleri, map keşfedilmemiş bölge asset'leri, DLC içerikleri, cutscene asset'leri.

Oyun İçi Kullanım Örneği

```
public class LazyItemIcon : MonoBehaviour {
    [SerializeField] private AssetReferenceSprite _iconRef;
    [SerializeField] private Image image;
    private AsyncOperationHandle<Sprite> _handle;

    // Sadece item görünür hale gelince tetiklenir
    public async void Show() {
        handle = _iconRef.LoadAssetAsync();
        await handle.Task;
        if (_handle.Status == AsyncOperationStatus.Succeeded)
            image.sprite = _handle.Result;
    }

    void OnDisable() {
        if (handle.IsValid()) Addressables.Release(handle);
    }
}
```

C. Addressables

Sprite Bundles (Atlas Organizasyonu)

Neden?	İlgili sprite'ları (UI ikonları, düşman atlas'ı, çevre tile'ları) aynı bundle'da gruplamak hem yükleme performansını hem draw call sayısını düşürür. Sprite atlas ile GPU batch sayısını azaltırsın.
Kullanmasam?	Her icon ayrı texture olursa yüzlerce draw call oluşur. Bundle olmadan granüler asset kontrolü yapamaz, gereksiz texture'lar bellekte kalır.
Nerede?	UI icon seti, bir bölümün tüm görsel asset'leri, karakter skin bundle'ları.

Oyun İçi Kullanım Örneği

```
// Addressables Groups Window:
// Weapons Bundle -> sword.png, bow.png, staff.png
// UI Icons Bundle -> health icon.png, mana icon.png
// FireLand Bundle -> tileset, bg_parallax, enemy_atlas

// Tüm bundle'ları aynı anda yükle
var handle = Addressables.LoadAssetsAsync<Sprite>(
    "UI Icons",
    sprite => _iconDict[sprite.name] = sprite
);
await handle.Task;

// Seviye yüklenirken FireLand bundle'ını önceden indir
var preload = Addressables.DownloadDependenciesAsync("FireLand");
await preload.Task;
Addressables.Release(preload);
```

D. Sheets Sync

Sheets API & CSV Export

Neden?	Düşman istatistikleri, item değerleri, diyalog metinleri gibi veriler tasarımcı/writer tarafından Sheets'te yönetilebilir. Programcı her değişiklik için kod açmaz.
Kullanmasam?	Tüm dengeleme verisi kod içinde ya da karmaşık Inspector alanlarında tutulur. Non-teknik ekip üyeleri veri giremez.
Nerede?	EnemyStats, ItemDatabase, DialogLines, LevelConfigs, LocalizationStrings.

Oyun İçi Kullanım Örneği

```
public static class SheetsSyncer {
    const string SHEET_URL =
        "https://docs.google.com/spreadsheets/d/YOUR_ID/export?format=csv&gid=0";

    [MenuItem("Tools/Sync Enemy Data")]
    static async void SyncEnemyData() {
        using var client = new System.Net.Http.HttpClient();
        string csv = await client.GetStringAsync(SHEET_URL);
        ParseAndApply(csv);
        AssetDatabase.SaveAssets();
        Debug.Log("[Sync] Enemy data updated!");
    }

    static void ParseAndApply(string csv) {
        var rows = csv.Split('\n');
        // rows[0] == header, rows[1+] == data
        foreach (var row in rows[1..]) { /* SO'yu güncelle */ }
    }
}
```

D. Sheets Sync

Auto Refresh (Only In Editor)

Neden?	Editor açıkken belirli aralıklarla veya dosya kaydetmede sheet'i otomatik çekmek, tasarımcı değişikliklerini anında oyunda test etmeyi sağlar.
Kullanmasam?	Her veri değişikliğinde manuel sync menüsüne tıklamak gerekir; insan hatası ile eski veri ile test yapılabilir.
Nerede?	Sadece Editor script olarak (EditorApplication.update veya FileSystemWatcher ile). Build'e dahil etme!

Oyun İçi Kullanım Örneği

```
[InitializeOnLoad]
public static class AutoSheetsRefresh {
    static double lastCheck;
    const double INTERVAL = 30.0; // saniye

    static AutoSheetsRefresh() {
        EditorApplication.update += OnEditorUpdate;
    }

    static void OnEditorUpdate() {
        if (EditorApplication.timeSinceStartup - _lastCheck < INTERVAL) return;
        _lastCheck = EditorApplication.timeSinceStartup;

        if (EditorApplication.isPlaying)
            SheetsSyncer.SyncEnemyData();
    }
}
```

E. Level Load

Additive Scenes

Neden? Ana gameplay sahnesi (Core) yüklenmiş kalırken bölüm sahneleri (FireLand, IceWorld) üstüne eklenir. HUD, GameManager gibi kalıcı nesneleri her sahnede tekrar koymazsın.

Kullanmasam? Her level değişiminde tüm sahne sıfırlanır: Singleton'lar yeniden kurulur, loading süresi uzar, flash screen oluşur.

Nerede? Core (kalıcı sistemler) + Level_X (bölüm geometri+enemy) + UI (overlay) şeklinde katmanlı yapı.

Oyun İçi Kullanım Örneği

```
public class SceneLoader : MonoBehaviour {
    private string _currentLevel;

    public async Task LoadLevel(string sceneName) {
        // Önce eski leveli kaldır
        if (!string.IsNullOrEmpty(_currentLevel)) {
            var unload = SceneManager.UnloadSceneAsync(_currentLevel);
            await unload;
        }
        // Yeni leveli additive yükle
        var load = SceneManager.LoadSceneAsync(sceneName, LoadSceneMode.Additive);
        await load;
        _currentLevel = sceneName;
        SceneManager.SetActiveScene(SceneManager.GetSceneByName(sceneName));
    }
}
```

E. Level Load

Async Loading & Loading Screen

Neden?	Async yükleme ana thread'i bloklamaz; loading ekranı, progress bar veya animasyon sorunsuz çalışmaya devam eder. allowSceneActivation ile yükleme tamamlanana kadar sahne geçişini geciktirebilirsin.
Kullanmasam?	Sync LoadScene = donma, kara ekran. Oyuncu 'crashed mi?' diye düşünür. Uzun seviyelerde 5-10 saniye tam donma olabilir.
Nerede?	Tüm level geçişlerinde, büyük asset bundle yüklemelerinde.

Oyun İçi Kullanım Örneği

```
IEnumerator LoadWithProgress(string scene) {
    _loadingScreen.SetActive(true);
    var op = SceneManager.LoadSceneAsync(scene, LoadSceneMode.Additive);
    op.allowSceneActivation = false; // Tam hazır olana kadar bekle

    while (op.progress < 0.9f) {
        _progressBar.value = op.progress;
        yield return null;
    }

    _progressBar.value = 1f;
    yield return new WaitForSeconds(0.3f); // Kısa bekleme: pürüzsüz UX
    op.allowSceneActivation = true;
    _loadingScreen.SetActive(false);
}
```

E. Level Load

Memory Cleanup

Neden?	Scene unload sonrası <code>Addressables.Release</code> + <code>Resources.UnloadUnusedAssets</code> + <code>GC.Collect</code> zinciri ile belleği temiz tutar. Uzun oyun sessionlarında memory leak önlenir.
Kullanmasam?	Sahneler boşaltılsa bile referans tutulan asset'ler bellekte kalır. Çok uzun oyunlarda veya mobilde crash kaçınılmaz olur.
Nerede?	Level transition, boss fight sonrası, menu'ye dönüşte.

Oyun İçi Kullanım Örneği

```
async Task CleanupAfterLevelUnload() {
    // 1. Addressable asset'leri serbest bırak
    foreach (var h in _levelHandles)
        if (h.IsValid()) Addressables.Release(h);
    _levelHandles.Clear();

    // 2. Kullanılmayan asset'leri boşalt
    var op = Resources.UnloadUnusedAssets();
    await op;

    // 3. Managed heap'i temizle
    System.GC.Collect();
    Debug.Log("[Memory] Cleanup complete.");
}
```

F. Animation Curves

Easing (Yumuşatma Eğrileri)

Neden?	Lineer hareket robotik görünür. AnimationCurve ile ease-in/ease-out uygulamak, UI animasyonlarını, kamera hareketlerini ve obje interpolasyonunu profesyonel hissettirir.
Kullanmasam?	Her şey sabit hızda hareket eder: açılan paneller mekanik, zıplayan düşmanlar robota benzer. Oyun 'cansız' hissettiriyor.
Nerede?	UI panel açılma/kapanma, kamera takibi, boss giriş animasyonu, skill indicator büyümesi.

Oyun İçi Kullanım Örneği

```
public class UIPanel : MonoBehaviour {
    [SerializeField] private AnimationCurve _openCurve; // ease out
    [SerializeField] private float _duration = 0.3f;

    public IEnumerator Open() {
        float t = 0;
        Vector3 target = Vector3.one;
        transform.localScale = Vector3.zero;

        while (t < _duration) {
            t += Time.deltaTime;
            float normalized = t / _duration;
            float curved = _openCurve.Evaluate(normalized);
            transform.localScale = Vector3.LerpUnclamped(Vector3.zero, target, curved);
            yield return null;
        }
        transform.localScale = target;
    }
}
```

F. Animation Curves

Stat Progression Curves (Stat İlerleme Eğrileri)

Neden? Oyuncu seviye 1'den 100'e giderken güç artışı lineer olmamalı. Curve ile erken seviyelerde hızlı ilerleyen, ileri seviyelerde yavaş ilerleme veya exponential büyüme tasarlayabilirsin.

Kullanmasam? Lineer formüle bağlı kalırsın. Balancing esnekliği yok, her seviyede aynı his.

Nerede? HP, damage, speed stat'larının level'a göre büyümesi; XP kazanım eğrisi; price scaling.

Oyun İçi Kullanım Örneği

```
[CreateAssetMenu(menuName="Game/StatCurve")]
public class StatCurveData : ScriptableObject {
    public AnimationCurve hpCurve;        // X: level (0-1), Y: çarpan
    public AnimationCurve damageCurve;
    public float baseHP      = 100f;
    public float baseDamage = 10f;

    public float GetHP(int level, int maxLevel) {
        float t = (float)level / maxLevel;
        return baseHP * hpCurve.Evaluate(t);
    }
}

// Lv1: 100 HP | Lv50: 800 HP | Lv100: 5000 HP (exponential curve)
```

F. Animation Curves

Movement Curves (Hareket Eğrileri)

Neden? Mermi yayı, düşman charge hareketi, kamera shake şiddetini AnimationCurve ile tanımlamak, hareket tasarımını koddan bağımsız ayarlanabilir kılar.

Kullanmasam? Hardcoded velocity değerleri. Mermi yayını değiştirmek için programcı çağırman gerekir.

Nerede? Mermi parabolü, ground slam hareketi, kamera shake, platforma zıplama.

Oyun İçi Kullanım Örneği

```
public class GroundSlam : MonoBehaviour {
    [SerializeField] private AnimationCurve _heightCurve; // zıplama yayı
    [SerializeField] private AnimationCurve _speedCurve; // hız profili
    [SerializeField] private float duration = 0.5f;

    IEnumerator SlamTo(Vector3 target) {
        float t = 0;
        Vector3 start = transform.position;

        while (t < _duration) {
            t += Time.deltaTime;
            float n = t / _duration;
            float height = _heightCurve.Evaluate(n);
            float speed = _speedCurve.Evaluate(n);
            Vector3 pos = Vector3.Lerp(start, target, speed);
            pos.y += height;
            transform.position = pos;
            yield return null;
        }
    }
}
```

F. Animation Curves

Spawn Pacing (Dalga Ritmi)

Neden? Düşman çıkış hızını AnimationCurve ile kontrol etmek, dalga başında yavaş, ortada yoğun, sonda boss şeklinde dramatik ritim yaratır.

Kullanmasam? Sabit spawn interval = monoton gameplay. Oyuncular dalgaları tahmin eder, gerilim kaybolur.

Nerede? WaveManager, bölüm boyunca spawn eden ambient düşmanlar, boss aşama geçişleri.

Oyun İçi Kullanım Örneği

```
public class WaveManager : MonoBehaviour {
    [SerializeField] private WaveData[] _waves;
    private int _waveIndex; private float _elapsed; private float _spawnBudget;

    void Update() {
        if (_waveIndex >= _waves.Length) return;
        var wave = _waves[_waveIndex];
        _elapsed += Time.deltaTime;

        // Curve: X=0..1 (dalga ilerlemesi), Y=spawn/sn
        float t = _elapsed / wave.waveDuration;
        float rate = wave.spawnRateCurve.Evaluate(t);

        _spawnBudget += rate * Time.deltaTime;
        while ( _spawnBudget >= 1f) {
            SpawnRandomEnemy(wave);
            _spawnBudget -= 1f;
        }
        if (_elapsed >= wave.waveDuration) NextWave();
    }
}
```

G. Save System

Binary Format Save

Neden?	JSON şeffaf ve hacklenebilir, PlayerPrefs ise çok sınırlıdır. BinaryWriter ile kayıt hem küçük hem okunaksızdır; cheating zorlaşır, I/O hızlanır.
Kullanmasam?	JSON save: oyuncular hex editör ile para/can düzenler. PlayerPrefs: karmaşık veri saklayamazsın, büyük save dosyaları için uygun değil.
Nerede?	Player progress, inventory, level completion, settings, achievement data.

Oyun İçi Kullanım Örneği

```
[System.Serializable]
public class SaveData {
    public int version = 1;
    public float playerHP; public int gold;
    public List<string> unlockedAbilities;
    public long saveTimestamp;
}

public static class SaveManager {
    static string Path => Application.persistentDataPath + "/save.dat";

    public static void Save(SaveData data) {
        data.saveTimestamp = DateTimeOffset.UtcNow.ToUnixTimeSeconds();
        using var fs = File.Open(Path, FileMode.Create);
        using var bw = new BinaryWriter(fs);
        var bytes = Encoding.UTF8.GetBytes(JsonUtility.ToJson(data));
        bw.Write(bytes.Length); bw.Write(bytes);
    }

    public static SaveData Load() {
        if (!File.Exists(Path)) return new SaveData();
        using var br = new BinaryReader(File.OpenRead(Path));
        return JsonUtility.FromJson<SaveData>(
            Encoding.UTF8.GetString(br.ReadBytes(br.ReadInt32())));
    }
}
```

G. Save System

Versioning & Migration

Neden?	Oyun güncellemelerinde save formatı değişebilir. Version numarası ile eski save'leri yeni formata migrate edebilirsin; oyuncuların ilerlemesi silinmez.
Kullanmasam?	Save formatı değişince eski kayıtlar corrupt olarak görünür. Oyuncuların ilerlemesi kaybolur = negatif store yorumları.
Nerede?	Save sistemi her major veri değişikliğinde version bump + migration fonksiyonu içermeli.

Oyun İçi Kullanım Örneği

```
public static SaveData Migrate(SaveData old) {
    switch (old.version) {
        case 0: // V0 -> V1: 'unlockedAbilities' alanı eklendi
            old.unlockedAbilities = new List<string> { "dash" };
            old.version = 1;
            goto case 1;
        case 1: // V1 -> V2: 'currencies' dict, 'gold'un yerini aldı
            // old.currencies = new Dict {{ "gold", old.gold }};
            old.version = 2;
            goto case 2;
        case 2: // güncel versiyon
            break;
    }
    return old;
}

// Yükleme sırasında hemen migrate et:
// var data = SaveManager.Load();
// data = Migrate(data);
```

G. Save System

Meta Data

Neden? Save dosyasının meta verisi (kayıt tarihi, play time, level, screenshot) oyunculara slot seçiminde bilgi sağlar ve cloud sync için gereken bağlamı verir.

Kullanmasam? Oyuncu hangi slot'u seçeceğini bilemez, yanlış slot üstüne yazar. Cloud sync hangi save'in daha yeni olduğunu belirleyemez.

Nerede? Save slot seçim ekranı, cloud sync conflict resolution, analytics.

Oyun İçi Kullanım Örneği

```
[System.Serializable]
public class SaveMeta {
    public int    version;
    public long   timestamp;           // Unix epoch
    public float  totalPlaySeconds;
    public int    currentLevel;
    public string heroName;
    public string screenshotBase64; // slot ekran görüntüsü
}

void DisplaySlot(SaveMeta meta) {
    var date = DateTimeOffset.FromUnixTimeSeconds(meta.timestamp);
    slotText.text = $"{meta.heroName} - Lv {meta.currentLevel}";
    dateText.text = date.ToString("dd MMM yyyy HH:mm");
    _timeText.text = TimeSpan.FromSeconds(meta.totalPlaySeconds)
        .ToString(@"hh\:mm");
}
```

H. Input System

New Unity Input System

Neden?	Eski Input.GetKey() yaklaşımı polling tabanlıdır ve çoklu cihaz desteği zahmetlidir. Yeni sistem event-driven'dır: InputAction ile gamepad, klavye, dokunmatik aynı kod ile çalışır.
Kullanmasam?	Gamepad desteği eklemek için kodu baştan yazman gerekir. Input rebinding (tuş değiştirme) eski sistemde çok karmaşıktır.
Nerede?	Player controller, UI navigation, debug cheats, replay sistemi.

Oyun İçi Kullanım Örneği

```
public class PlayerInput : MonoBehaviour {
    private GameControls _controls;

    void Awake() {
        _controls = new GameControls();
        _controls.Player.Jump.performed += OnJump;
        _controls.Player.Attack.performed += OnAttack;
        _controls.Player.Move.performed += OnMove;
    }
    void OnEnable() => _controls.Enable();
    void OnDisable() => _controls.Disable();

    void OnJump(InputAction.CallbackContext ctx) {
        _playerController.Jump();
    }
    void OnMove(InputAction.CallbackContext ctx) {
        Vector2 dir = ctx.ReadValue<Vector2>();
        _playerController.SetMoveDirection(dir);
    }
}
```

H. Input System

Gamepad & Keyboard Desteęi

Neden?	InputActionAsset'te aynı Action'a hem WASD hem gamepad sol stick bind etmek, platform farkı olmaksızın her ikisini de destekler. Control scheme ile hangi cihaz aktif olduğunu anlayabilirsiniz.
Kullanmasam?	Sadece klavye için yazılmış oyun: gamepad'i takınca hiçbir şey çalışmaz. Console/mobile port'ta sıfırdan input yazman gerekir.
Nerede?	PC + console release hedefleyen her proje. Mobil için Touch Screen de aynı Action'a bind edilebilir.

Oyun İçi Kullanım Örneęi

```
public class PlayerController : MonoBehaviour {
    private Vector2 _moveInput;

    // Input System callback'leri
    public void OnMove(InputValue value) => _moveInput = value.Get<Vector2>();
    public void OnJump(InputValue value) {
        if (value.isPressed) DoJump();
    }

    // Aktif cihazı algıla
    public void OnControlsChanged(UnityEngine.InputSystem.PlayerInput pi) {
        bool isGamepad = pi.currentControlScheme == "Gamepad";
        _hud.ShowGamepadIcons(isGamepad);
    }
}
```

I. Quick Test

Herhangi Ortam / Stage'den Başlat

Neden? Level 10'daki bir bug'ı test etmek için 1-9 arasını oynamak zaman kaybıdır. Debug launcher ile direkt 500m Fire Land'e spawn olabilirsiniz.

Kullanmasam? Her test döngüsünde baştan oynamak gerekir. Geç game content testleri neredeyse imkansız hale gelir.

Nerede? Editor-only DebugLauncher scripti. Build'e dahil etme, platform direktifi kullan.

Oyun İçi Kullanım Örneği

```
#if UNITY_EDITOR
[CreateAssetMenu(menuName="Debug/LaunchConfig")]
public class DebugLaunchConfig : ScriptableObject {
    public string targetScene    = "FireLand";
    public float  startDistance = 500f;
    public string heroId        = "warrior";
    public int    startGold     = 9999;
    public bool   enableDebugHUD = true;
}

[InitializeOnLoad]
public static class QuickLauncher {
    static QuickLauncher() {
        EditorApplication.playModeStateChanged += OnPlay;
    }
    static void OnPlay(PlayModeStateChange state) {
        if (state != PlayModeStateChange.EnteredPlayMode) return;
        var cfg = Resources.Load<DebugLaunchConfig>("DebugLaunchConfig");
        if (cfg != null) GameBootstrapper.StartWithConfig(cfg);
    }
}
#endif
```

I. Quick Test

Inventory Override & Zaman Kontrolü

Neden?	Belirli item kombinasyonu ile boss'u test etmek ya da gece modunu test etmek için oyunu belirli bir state'te başlatabilmek kritiktir.
Kullanmasam?	Her kombinasyonu oyun içinde toplamak saatler alır. Zaman bazlı mekanikleri test etmek neredeyse imkansız olur.
Nerede?	Debug launcher + CheatConsole (sadece editor/dev build'de aktif).

Oyun İçi Kullanım Örneği

```
public class CheatConsole : MonoBehaviour {
    void Update() {
#if DEVELOPMENT_BUILD || UNITY_EDITOR
        if (Input.GetKeyDown(KeyCode.F1)) GiveAllItems();
        if (Input.GetKeyDown(KeyCode.F2)) SetGameTime(22, 0); // gece
        if (Input.GetKeyDown(KeyCode.F3)) Time.timeScale = 5f; // x5 hız
        if (Input.GetKeyDown(KeyCode.F4)) KillAllEnemies();
#endif
    }

    void SetGameTime(int hour, int minute) {
        GameManager.Instance.SetTime(hour, minute);
        Debug.Log($"[Cheat] Game time set to {hour:00}:{minute:00}");
    }
}
```

I. Quick Test

Stress Testing

Neden? Sahneye 100+ düşman, 1000 mermi koyarak bottleneck'leri erken tespit edersin. Unity Profiler ile hangi sistemin darboğaz oluşturduğunu bulursun.

Kullanmasam? Performans sorunları sadece oyuncu şikayeti ile öğrenilir. Late-stage optimization çok daha maliyetlidir.

Nerede? Editor-only stress test sahnesi; CI pipeline'ında otomatik performans baseline testi.

Oyun İçi Kullanım Örneği

```
#if UNITY_EDITOR
public class StressTest : MonoBehaviour {
    [SerializeField] private int _enemyCount = 200;
    [SerializeField] private int _projectileRate = 50; // /sn

    [ContextMenu("Run Stress Test")]
    void Run() {
        StartCoroutine(SpawnEnemies());
        StartCoroutine(FireProjectiles());
    }

    IEnumerator SpawnEnemies() {
        for (int i = 0; i < _enemyCount; i++) {
            EnemyFactory.Instance.Create("goblin",
                Random.insideUnitSphere * 20f);
            if (i % 10 == 0) yield return null; // frame'e yay
        }
    }
}
#endif
```

J. Localization

Unity Localization System

Neden? Unity'nin resmi Localization paketi, String/Asset table'lar ile metni ve görselleri dilden bağımsız yönetir. Runtime'da dil değişimi, Google Sheets sync ve plural rules desteği sunar.

Kullanmasam? Homemade string dict ile başlarsın, büyüdükçe yönetilemez hale gelir. Plural (1 elma / 3 elma), RTL dil desteği, font override gibi özellikler yoktur.

Nerede? Tüm UI metinleri, item isimleri, diyaloglar, achievement açıklamaları.

Oyun İçi Kullanım Örneği

```
public class LocalizedLabel : MonoBehaviour {
    [SerializeField] private LocalizedString _key;
    [SerializeField] private TMP_Text _text;

    void OnEnable() => _key.StringChanged += UpdateText;
    void OnDisable() => _key.StringChanged -= UpdateText;
    void UpdateText(string value) => _text.text = value;
}

// Runtime dil değiştirme:
// using UnityEngine.Localization.Settings;
// LocalizationSettings.SelectedLocale =
//     LocalizationSettings.AvailableLocales.GetLocale("tr");
```

J. Localization

Key Naming: domain.object.field

Neden? Binlerce localization key'i olan projede isimlendirme standardı olmadan arama yapılamaz, duplikasyon artar. domain.object.field formatı hiyerarşik tarama sağlar.

Kullanmasam? btn1, text_23, msg_attack gibi anlamsız key'ler. Dil dosyası büyüdükçe context kaybolur ve yanlış metinler kullanılır.

Nerede? Tüm string table key'leri bu formata uymalı. Otomasyon scriptleri de aynı formatı kontrol etmeli.

Oyun İçi Kullanım Örneği

```
// Format: domain.object.field
// domain -> ui, gameplay, item, enemy, hud, menu, dialog
// object -> ekrandaki element
// field -> label, desc, tooltip, title, button, error

// ui.mainmenu.start button -> 'Oyuna Basla'
// ui.mainmenu.settings_button -> 'Ayarlar'
// ui.hud.health label -> 'Can'
// ui.shop.buy button -> 'Satın Al'
// gameplay.ability.dash.name -> 'Hızlı Kosu'
// gameplay.ability.dash.desc -> '{0} saniyede {1} birim kosar'
// item.sword_iron.name -> 'Demir Kılıç'
// item.sword_iron.desc -> 'Basit ama güvenilir bir silah.'
// enemy.goblin.name -> 'Goblin'
// dialog.npc.blacksmith.intro -> 'Merhaba, silah mı istiyorsun?'
// error.save.failed -> 'Kayıt başarısız oldu.'
```